MPS-Fuzz: An Enhanced Fine-Grained Fuzzing Based on Units With Multiple Inputs and Outputs

Ximing Fan, Yong Fang, Peng Jia, Hongwei Li, *Fellow, IEEE*, Xi Peng, Yijia Xu, Qinying Wang, Binbin Zhao, and Shouling Ji, *Member, IEEE*

***Abstract*—Edge coverage-guided. fuzzing has demonstrated remarkable achievements in vulnerability discovery. Some studies with fine-grained coverage metrics have been proposed to enhance the vulnerability mining capabilities of fuzzing by capturing more program paths. However, this refinement often results in a significant increase in seeds, which are highly homogeneous and may limit vulnerability detection. Additionally, finer granularity requires more bitmap hits, increasing the risk of hash collisions. To address these shortages, the paper proposes the structure of a basic block unit with multiple predecessors and successors (referred to as MPS). Then, a fine-grained coverage method called MPS-Fuzz is designed based on the MPS structure. In this approach, it is convenient to exclude basic blocks involving loop structures when determining MPS units, which helps reduce seed homogeneity. Additionally, we introduce an additional bitmap to record the coverage status of MPS units, ensuring that the collision rate of the edge bitmap does not increase. Moreover, these additional operations do not incur excessive time overhead. To demonstrate the properties of the MPS-Fuzz, we implement our approach on AFL and conduct experiments on 16 benchmarks from FuzzBench and Unifuzz. The result indicates that, after 24-hour fuzzing, MPS-Fuzz explores an average of 9.6% more edges and an average of 25.7% more bugs than AFL. Compared to other fine-grained coverage methods (N-gram and PathAFL), MPS-Fuzz also achieves better performance. Moreover, MPS-Fuzz has discovered a previously unknown bug on real-world program and got a CVE assigned**

***Index Terms*—Grey box fuzzing, fine-grained coverage, path sensitive, distinct instrumentation, MPS unit, loop structure.**

I. Introduction

GREY box fuzzing is a widely used technology for finding vulnerabilities. Within grey box fuzzing, code coverage serves as a guiding principle to filter out the more valuable cases from generated extensive test cases. Notably, it is not necessary to know the functionality and logic of the target. Fuzzers could leverage the structure information (e.g., control flow graph, referred CFG) to obtain coverage for bug finding. To date, CFG information (mainly refers to edge coverage) has become almost indispensable for various types of fuzzers, such as library API fuzzer [1], [2], [3], protocol fuzzer [4], [5], database fuzzer [6], [7]. The programming languages involved are also diverse, including compiled language fuzzer AFL [8], interpretive language fuzzer [9], [10], and even cross language fuzzer [11].

Although the edge coverage-guided fuzzing has shown effectiveness, it overlooks many program execution states. For instance, in protocol fuzzing, relying solely on edge coverage fails to capture the transitions of protocol states [4], [5], [12]. Focusing solely on control flow coverage, edge coverage guidance still ignores some program execution paths [13], [14], [15]. Taking AFL as an example, it designs a data structure, i.e., bitmap, to record the edge hits during program execution, which has been inherited by almost all fuzzers today. However, the emergence of a new path does not necessarily lead to the emergence of a new edge. It means that the edge coverage can never discover some execution paths. To record more execution paths, fine-grained coverage (e.g., N-gram [13] and PathAFL [15]) aggregates more basic block IDs through hashing to mark paths. Hashing a sequence of basic block IDs is indeed a direct and effective solution, but it also introduces some issues. Firsly, for loop structures of program, fine-grained coverage will capture execution paths under many different loop counts. While the edge coverage only identifies two distinct paths: one that executes a loop structure and another that does not. However, capturing additional paths with varying loop iterations does not significantly aid vulnerability discovery. Due to the lack of data flow information, increasing the number of loop iterations makes it difficult to reach the specific threshold needed to trigger a bug. Fine-grained coverage methods largely overlook the impact of new paths introduced by loop structures, wasting considerable time on these paths. Secondly, to discover more execution paths under complex topological structures, additional operations such as more complex instrumentation and seed scheduling logic are required. These operations often result in increased bitmap hash collisions and time overhead. These two points will be discussed in more detail through specific code snippets in Section II. Therefore, an enhanced fine-grained coverage method is challenging

Received 27 October 2024; revised 17 September 2025; accepted 2 December 2025. Date of publication 15 December 2025; date of current version 12 March 2026. This work was supported by Sichuan Provincial Science Foundation under Grant 2024YFHZ0015 and in part by the Science Challenge Project under Grant TZ2025002. *(Corresponding authors: Peng Jia; Yong Fang.)*

Ximing Fan, Yong Fang, Peng Jia, Xi Peng, and Yijia Xu are with the School of Cyber Science and Engineering, Sichuan University, Chengdu 610065, China (e-mail: fanximing@stu.scu.edu.cn; yfang@scu.edu.cn; pengjia@scu.edu.cn; pengxi@stu.scu.edu.cn; xuyijia@scu.edu.cn).

Hongwei Li is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 610065, China (e-mail: hongweili@uestc.edu.cn).

Qinying Wang and Shouling Ji are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: wangqinying@zju.edu.cn; sji@zju.edu.cn).

Binbin Zhao is with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: binbin.zhao@gatech.edu).

Digital Object Identifier 10.1109/TDSC.2025.3641553

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

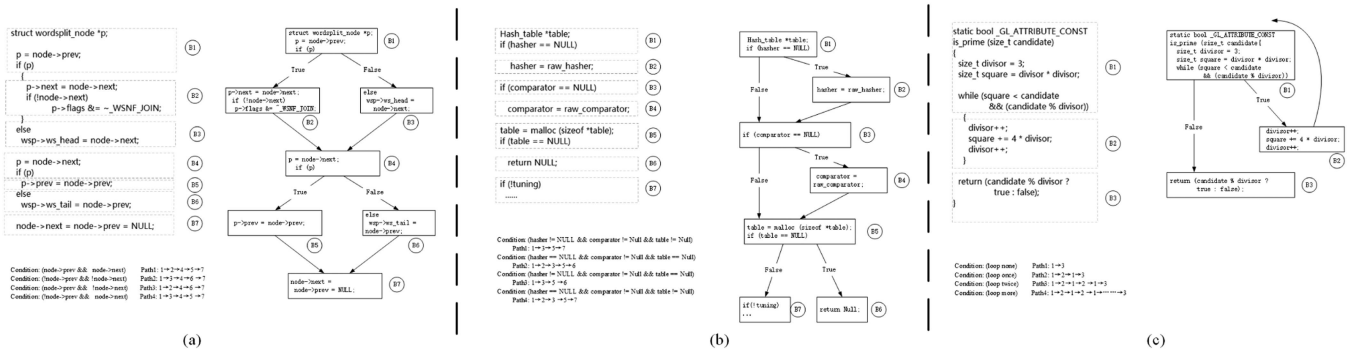

Fig. 1. Three causes of "path lost". Each scenario is illustrated with source code (all the codes are from the 'cflow' project), a CFG diagram, and possible execution paths. (a) the superposition of two conditional judgments; (b) the superposition of three or more conditional judgments; (c) scenarios involving loop structures.

from two perspectives finding: **Challenge 1,** how to filter the additional paths captured by fine-grained coverage, particularly those stemming from loop bodies. **Challenge 2,** how to balance paths discovery and the cost of collision and time overhead increased.

To address these challenges, the paper introduces a fine-grained coverage method called MPS-Fuzz, based on the MPS unit. An MPS unit is a special structure in a program's CFG that has multiple predecessors and successors and can consist of a single basic block or a combination of several basic blocks. The MPS unit could be used to explain the relationship between program execution paths and fuzzer's coverage metric (more details could be found in Section II). The method identifies MPS units and basic blocks related to loop structures through static analysis during compilation process and categorizes the basic blocks into different types. It is convenient to exclude basic blocks involving loop structures when determining MPS units, which helps reduce seed homogeneity. Therefore, the method has advantages in enhancing code coverage and discovering bugs. Subsequently, different types of basic blocks are instrumented differently. When code executing, the instrumentation codes maintain not only an edge coverage bitmap but also an MPS-unit coverage bitmap. As the additional instrumentation primarily targets basic blocks within the MPS structure, this method discovers more paths while avoiding excessive time overhead and bitmap collisions. Specifically, the main contributions of the paper are as follows:

- To the best of knowledge, we are the first to unveil the issue of seed homogenization caused by loop structures in fine-grained coverage methods. By excluding loop structures during static analysis, we provide a solution to this problem;
- A novel structure of the MPS unit has been proposed, and a fine-grained coverage method called MPS-Fuzz has been designed based on the MPS structure, which could capture

II. Motivation

The bitmap is used to store coverage information, which accelerates the retrieval and storage of unique paths, greatly enhancing the fuzzer's throughput. However, the hits in the bitmap, which represent local sequences of basic blocks, may overlap, leading to some paths never being discovered. We refer to this phenomenon as "path lost". Fig. 1(a) shows an example from an actual program. It shows four execution paths, dominated by variables 'node->prev' and 'node->next'. Typical edge coverage can only capture at most two paths (e.g., P1 and P2) due to overlap in edge hits.

The above example is a typical case of "path lost" caused by a stack of two conditional statements, which is also the motivation for proposing finer-grained coverage in references [13], [14], [15], [18]. However, "path lost" can stem from various scenarios beyond this. The paper identifies two additional situations, as shown in Fig. 1(b) and (c). Scenario (b) involves the superposition of three (or even more) conditional judgments, potentially leading to even more paths than Fig. 1(a). Moreover, it cannot be simply viewed as a superposition of two instances of (a). While some fine-grained coverage methods may address (a), they might overlook paths in (b). Scenario (c) involves loop structures, where variations in loop iterations primarily generate different paths. In all scenarios in diagram, for edge coverage, if Path1 and Path2 are discovered, then Path3 and Path4 cannot be detected.

The paper, through observation of the CFG's topological structures under the aforementioned three scenarios, identifies that the main cause of the "path lost" issue is the basic block unit with multiple predecessors and successors. The unit can be a single basic block or a combination of multiple basic blocks. As shown in Fig. 2, the three types of MPS unit correspond to the three scenarios presented in Fig. 1.

Not only does edge coverage experience the problem of "path

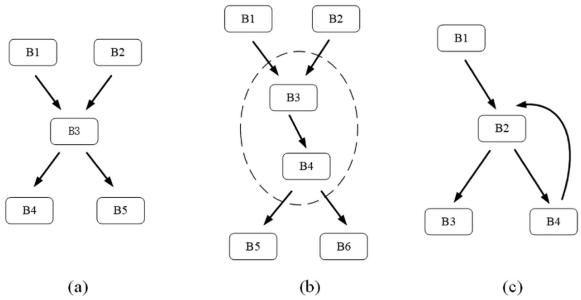

Fig. 2. Three types of MPS unit. (a) The MPS unit (B3) is a single basic block. (b) The MPS unit (B3 and B4) is a combination of multiple basic blocks. (c) The MPS unit (B2) is the single basic block that is the head of the loop structure.

all the hits of coverage bitmap that constitute $\mathcal{P}$}. For a particular path p, b={a set of all the basic blocks that constitute p}, h={a set of all the hits of coverage bitmap that constitute p}.

- *Criteria for "path lost":* If $\exists\, p \notin \mathcal{P}$, $b \subsetneqq \mathcal{B}$, and $h \subsetneqq \mathcal{H}$, then p cannot be detected, in the current coverage measurement;
- *Inference:* The MPS unit introduces the issue of "path lost", and employing fine-grained coverage can address this problem. For more complex MPS units (i.e., those with a greater number of basic blocks) or combinations of various MPS units, capturing more program paths requires a more fine-grained coverage method.

In practical testing, it is neither possible nor necessary to record all possible execution paths. Fuzzers of different granularities exhibit varying degrees of "path lost", that is, the number of paths discovered differs. For different granularities of coverage-guided methods, the more representative works include Honggfuzz [19], which uses basic block coverage; AFL, which records the hit of the edge between two basic blocks; and N-gram, which records the hash value of a sequence of basic blocks with a variable length. Honggfuzz, AFL, and N-gram gradually refine the granularity of coverage, enabling the exploration of more program paths. Ideally, the N-gram could be regarded as path coverage if 'N' is infinite. Nevertheless, in practice, excessive 'N' will result in path explosion, unacceptable time overhead, and collision.

The paper's motivation primarily stems from an understanding and contemplation of the N-gram method and practical issues encountered in experiments. The N-gram method provides an intuitive and effective means of achieving fine-grained coverage by combining the IDs of n basic blocks through hashing (specifically, $shift$ and xor operations). The IDs of n basic blocks in a sequence are presented as a vector (block[1], block[2],..., block[n]). Formula (1) describes the hash method used for calculating edge coverage in AFL, while Formula (2) outlines the hashing method for basic block sequence IDs in N-gram. Notably, Formula (2) details the N-gram hashing approach in AFL++[20], which slightly differs from the original version. The effectiveness is nearly identical, but the implementation is more straightforward.

$$\text{Map\_idx} = (\text{block}[1] >> 1) \oplus \text{block}[2] \tag{1}$$

$$\text{Map\_idx} = (\text{block}[1] >> 1) \oplus \ldots \oplus (\text{block}[n-1] >> 1) \oplus \text{block[n]} \tag{2}$$

Referring to the three scenarios in Fig. 2, when 'N' is set to 2 (i.e., hashing three basic block IDs), it can address the scenario in Fig. 2(a); for scenario 2(b), where the MPS unit contains more basic blocks, a larger 'N' is required ('N' is at least 4). In the case of Fig. 2(c), a larger 'N' means capturing more iterations of loop body. Since the N-gram performs the same hashing operation on each basic block, an increase in 'N' significantly enhances the ability to capture execution paths (manifested by a notable increase in the number of seeds). However, this also leads to increased time overhead and bitmap density [20], which are detrimental to vulnerability mining.

Given that some special structures in the CFG (i.e., the scenarios of MPS unit in Fig. 2) require fine-grained coverage methods. In contrast, some simpler structures do not need more granular coverage than edge coverage. Performing hash operations on basic block sequences only at these special structures (as done in N-gram) can capture more execution paths while also reducing time overhead and bitmap density, which helps to address Challenge 2. Additionally, employing distinct instrumentation for different basic blocks, as opposed to the globally consistent instrumentation approach of N-gram, facilitates targeted handling of loop structures, which helps to address Challenge 1. These special basic blocks can be identified through static analysis during the program compilation process. Thanks to the analysis capabilities of LLVM and Clang, it is possible to build adjacency matrices for predecessors and successors of basic blocks. The matrices can be used to identify these particular structures.

III. Method

This section elaborates on the details of the proposed MPS-Fuzz, which establishes an additional bitmap for storing and retrieving MPS unit information to supplement the edge coverage bitmap in discovering more program execution paths. First, we introduce the overall process of the method, specifically the role of MPS-Fuzz during certain stages of the fuzzing process. Next, we detail the static analysis and code instrumentation procedures. To streamline discussions, we denote specific basic block hashing operations as $hash(a, b, \ldots)$ and n-basic block hashing as $n - hash$, with the path consisting of a sequence of basic blocks denoted as (a, b, c,...). "Loop structure header" refers to the starting basic blocks of loop structures in the CFG.

A. Framework of MPS-Fuzz

The MPS-Fuzz measures a test case using a tuple (edge coverage, MPS unit coverage). A test case is saved if there is a new hit in either the edge bitmap or the MPS-unit bitmap. Edge coverage is the same as AFL. MPS unit coverage can be described by Formula (3), where the 'multi-input vector' refers to a vector that stores the input edges of MPS units. The core of Formula is that when executing a basic block with multiple predecessors, the vector is updated, and when executing a basic block with multiple successors, the MPS unit index is updated by hashing.

$$\begin{cases} Multi\_input\_vector >> 1 & \text{if edge is the MPS input} \\ Multi\_input\_vector \oplus edge & \text{if edge is the MPS output} \end{cases} \tag{3}$$

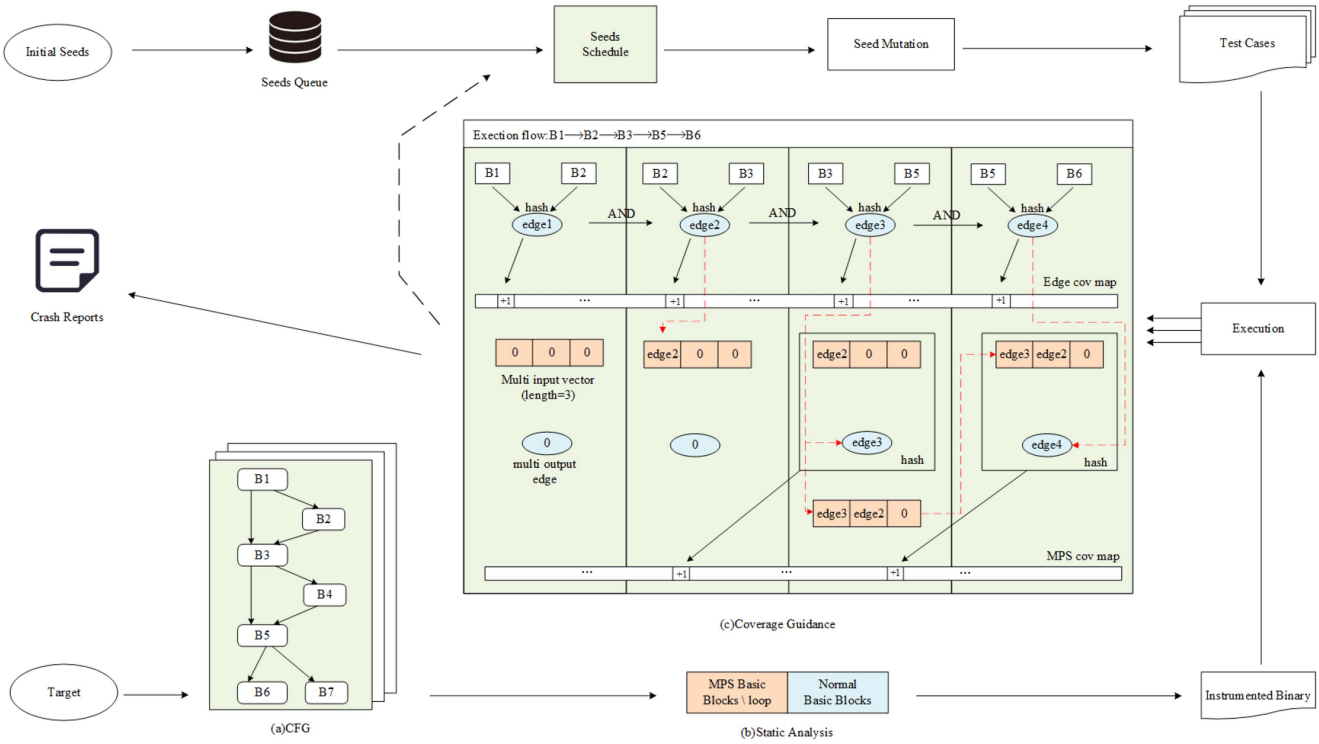

Fig. 3. Workflow diagram of MPS-Fuzz. The colored portions in the flowchart represent the method proposed in the paper to the steps of the fuzzing process. (a) The CFG of a program slice, where $B*$ is the basic block and arrows represent the direction of program execution. (b) is the result of static analysis using the CFG, which classifies basic blocks into two significant categories: MPS basic blocks and normal basic blocks. For a particular execution path (B1, B2, B3, B5, B6), (c) is an example of MPS-Fuzz's map updating process.

The framework of MPS-Fuzz is shown in Fig. 3. In the compilation phase, attributes of basic blocks are initially required by static analysis, serving as prior knowledge to guide code instrumentation. Specifically, the static analysis process categorizes basic blocks into normal and MPS basic blocks. Within the MPS basic blocks, some are identified as loop structure headers, yet these blocks are also classified as normal basic blocks. For normal basic blocks, conventional instrumentation is employed to calculate edge coverage. Additional instrumentation is required to calculate MPS unit coverage for MPS basic blocks. During program execution, edge coverage and MPS unit coverage information are stored in separate 64 KB bitmaps. As mentioned earlier, an MPS unit is a structure with multiple inputs and outputs. The paper aggregates the input and output edges of the MPS unit through a hash operation to form a unique ID representing this unit. This ID is then stored in the MPS coverage map. In actual program execution, a path may contain numerous basic blocks, leading to the presence of many MPS units. These MPS units vary in size (i.e., the number of constituent basic blocks) and may overlap. To address the complexity, the paper employs a vector called the 'multi-input vector' to store the input edges of MPS units. This vector enables the simultaneous recording of multiple MPS units, with the length of the vector pre-set to determine the number of units recorded (e.g., the vector length in Fig. 3 is set to 3). Moreover, this multi-input vector is continuously updated, meaning each multi-output edge undergoes a hash operation with the three closest multi-input edges, allowing the monitoring of all paths across three consecutive MPS units. The basic block sequence covered by three consecutive MPS units comprises at least five basic blocks, with more basic blocks in practical scenarios. Hence, this method possesses enhanced path coverage capabilities.

The MPS-Fuzz principle is exemplified through a concrete scenario. In Fig. 3, to capture the path (B1, B2, B3, B5, B6), edge coverage carries out 4 edges through the hash steps $hash(B1, B2)$, $hash(B2, B3)$, $hash(B3, B5)$ and $hash(B5, B6)$. In these four steps, the multi-input vector is updated when the flow reaches the input edge of the MPS unit. When the flow reaches the output edge of the MPS unit, the multi-input vector is hashed with the multi-output edge to obtain the MPS unit ID. Specifically, in the first step, the multi-input vector and the multi-output edge are initialized to zero. Since edge1 does not belong to the MPS unit, no extra action is taken. In the second step, edge2 is the input of the MPS unit, so the multi-input vector is updated. In the third step, edge3 serves as both the input of the MPS unit and the output of another MPS unit. Therefore, the multi-input vector is hashed with the multi-output edge, then the multi-input vector is updated. In the fourth step, edge4 is the output of the MPS unit, so the multi-input vector is hashed with the multi-output edge. Here, the calculation method for hashing the multi-input vector with the multi-output edge is consistent with Formula (2). Notably, MPS-Fuzz executes additional operations only at specific basic block positions, facilitating the discovery of potential paths more

efficiently compared to the N-gram method, which requires operations at every basic block position.

MPS-Fuzz filters the loop structure headers, reducing the instrumentation of related basic blocks. This prevents redundant loop paths from generating new hits of MPS unit bitmap, thereby reducing the discovery of redundant paths. Take Fig. 2(c) for example. Path "P1: (B1,B2,B3)", it does not go through any loops, "P2: (B1,B2,B4,B2,B3)", it goes through 1 loop, and "P3: (B1,B2,B4,B2,B4, B2,B3)", it goes through 2 loops. MPS-Fuzz, like AFL, can capture at most 2 paths, while N-gram can capture all 3 paths. MPS-Fuzz uses two bitmaps: one for edge coverage and one for MPS unit coverage. The edge coverage can capture at most 2 paths (e.g. (P1, P2) or (P1 P3)), while the MPS unit coverage captures 0 path due to the filtering of loop headers. N-gram can still capture P3 after capturing P1 and P2 due to the new hit hash(B4,B2,B4). In this simple example, there are only 3 paths, but if there are more basic blocks in the loop structure, N-gram will capture more paths, while MPS-Fuzz will still capture at most 2 paths.

B. Static Analysis and Instrumentation

The static analysis identifies basic blocks associated with the MPS unit while excluding loop structures, as outlined in Algorithm 1. It starts by traversing each function of the target to gather loop-related information (lines 3-4). Then, it iterates through each basic block within the function, assigning a label to each block. Based on the number of predecessors and successors, the algorithm determines whether a basic block serves as an input, output, both, or neither of an MPS unit (lines 14, 17, and 21). Lastly, the algorithm filters each basic block's label using loop information. If a basic block or any of its predecessors starts a loop structure, its attribute is redefined as unrelated to the MPS unit (lines 23-26).

The MPS-Fuzz's instrumentation is shown in Algorithm 2. The customizing instrumentation for each basic block based on its label. First, the algorithm initializes a multi-input vector to track the input edges of the MPS unit. Using the edge coverage method, each edge's ID is recorded and stored in a bitmap (lines 4-11). Following this, additional instrumentation is applied based on the labels of basic blocks obtained from Algorithm 1. Specifically, the approach targets three types of special basic blocks (lines 13-23):

a) For a basic block that serves as both an input and output to the MPS unit, the process begins by hashing the multi-input vector with the current edge to obtain the MPS unit ID, which is then recorded in another bitmap. Subsequently, the multi-input vector is updated with the current edge. The update method described here can be seen as a right shift of vector elements.

b) The multi-input vector is updated with the current edge for a basic block that is solely an input to the MPS unit.

c) For a basic block solely an MPS unit output, the process involves hashing the multi-input vector with the current edge to obtain the MPS unit ID, which is then stored in the MPS coverage bitmap.

Algorithm 1: Static Analysis for Determining MPS Units.

Input: Program under testing, PUT
Output: Basic blocks' labels
1 $\mathcal{F} \leftarrow$ Set of all functions from PUT;
2 $\mathcal{B} \leftarrow$ Set of all basic blocks from a certain function of $\mathcal{F}$;
3 **for** *function* $f \in \mathcal{F}$ **do**
4 $\mathcal{LH} \leftarrow$ Set of all loop structure starting basic blocks within f;
5 /* Determining the basic blocks' labels */ ;
6 **for** *block* $b \in \mathcal{B}$ **do**
7 /* predecessor(*) denotes all the predecessors of one basic block */ ;
8 $\mathcal{P}$ = predecessor(b);
9 // | * | means the number of elements in a set ;
10 **if** $|\ \mathcal{P}\ | > 1$ **then**
11 /* successor(*) denotes all the successors of one basic block */ ;
12 **if** $\exists p \in \mathcal{P}$,*and* $|\ success(p)\ | > 1$ **then**
13 /* b is both input and output of a MPS unit;
14 b->label = 'MIMO';
15 **else**
16 /* b is the input of a MPS unit */;
17 b->label = 'MI';
18 **else**
19 **if** $|\ success(p)\ | > 1,\ p \in \mathcal{P}$ **then**
20 /* b is the output of a MPS unit */;
21 b->label = 'MO';
22 /* filtering the loop structure */ ;
23 **for** *block* $b \in \mathcal{B}$ **do**
24 **if** $b \in \mathcal{LH}$ *or* $\exists predecessor \in \mathcal{LH}$ **then**
25 /* Fix b's label to normal */;
26 delete b->label;
27 **return** Basic blocks' labels

IV. Experiment

This section details the extensive evaluation on MPS-Fuzz to demonstrate its effectiveness in improving fuzzing performance. There are six research questions that will be answered.

RQ1. The filtering capability of MPS-Fuzz on paths inspired by loops;
RQ2. Effectiveness in increasing edge coverage;
RQ3. Effectiveness in discovering bugs;
RQ4. Time overhead and collisions evaluation;
RQ5. Ablation experiments;
RQ6. Scalability of MPS-Fuzz;

A. Experimental Setup

The paper implements MPS-Fuzz using AFL and its $llvm\_mode$. We randomly selected 16 benchmark programs,

Algorithm 2: Instrumentation for MPS Unit Coverage Map Updating.

Input: Program under testing, PUT
Output: Instrumented program, $Instr\_PUT$

```
B ← Set of all basic blocks from PUT;
/* Initializing multi-input vector and previous bb */ ;
V = [0,0,0] ;
b_p_idx = 0 ;
/* Instrumentation */ ;
for block b ∈ B do
    /* Instrumentation of edge coverage */ ;
    b_idx = random(MPA_SIZE) ;
    Edge = (b_idx >>1) ⊕ b_p_idx ;
    Edge_cov_map[Edge]++ ;
    b_p_idx = b_idx ;
    /* Instrumentation of MPS unit coverage */ ;
    if b->label = 'MIMO' then
        Val = (V[0]>>1)⊕(V[1]>>1) ⊕(V[2]>>1) ;
        Key = (Val>>1) ⊕ Edge ;
        MPS_cov_map[Key]++ ;
        V = [ Edge, V[0], V[1] ] ;
    if b->label = 'MI' then
        V = [ Edge, V[0], V[1] ] ;
    if b->label = 'MO' then
        Val = (V[0]>>1)⊕(V[1]>>1) ⊕(V[2]>>1) ;
        Key = (Val>>1) ⊕ Edge ;
        MPS_cov_map[Key]++ ;
return Instr_PUT
```

TABLE I
BENCHMARK DETAILS

Target	Source file	Input format	Number of seed	Basic blockes	Version
harfbuzz	harfbuzz	ttf,otf	472	148k	7.3.0
libpng	libpng	png	169	9k	1.6.4
libxml2	libxml2	xml	318	58.6k	2.10.1
sqlite3	sqlite3	text	1263	109k	3.33.0
freetype	freetype2	ttf,otf	2	40k	2.13.2
openssl	openssl	binary	1371	123k	3.1.2
lcms	Little-CMS	profile	1	16k	2.15
bloaty	bloaty	binary	101	289k	1.1
tcpdump	tcpdump-4.8.1	binary	100	6k	4.8.1
nm	binuitls-5279478	binary	32	38k	2.28
mp4	Bento4-1.5.1-628	mp4	100	33k	1.5.1
mp3	mp3gain-1.5.2	mp3	100	2k	1.5.2
tiffsplit	libtiff-3.9.7	tif	100	7k	3.9.7
cflow	cflow-1.6	text	100	6k	1.6
flvmeta	flvmeta-1.2.1	flv	100	7k	1.2.1
lame	lame-3.99.5	wav	100	8k	3.99.5

eight from FuzzBench and eight from UniFuzz datasets. Program details are shown in Table I, with the first eight targets from FuzzBench and the rest from UniFuzz.

To maximize bug triggering in the experiments, ASAN [21] (AddressSanitizer) instrumentation was applied to all targets. Alongside AFL, we chose N-gram and PathAFL as baseline comparisons due to their fine-grained coverage capabilities. Given the variable 'N' in N-gram, values of 2, 4, and 8 were selected as recommended by N-gram and AFL++. These cases are denoted as N-gram(2), N-gram(4), and N-gram(8) in subsequent sections. Given that AFL++ optimized N-gram, we decided to use the N-gram code from AFL++ for our experiments. The size of the multi-input vector in our method is 3 (the performance under different sizes will be shown in subsequent ablation experiments). Due to its support for only GCC instrumentation, PathAFL cannot compile fuzzbench projects as compiling fuzzbench harness requires linking with a library based on LLVM instrumentation. Therefore, only data from partial projects are displayed for PathAFL.

Experiments were conducted on two servers equipped with Intel(R) Xeon(R) E5-2680 v4 56-Core 2.40 GHz CPU, 128 GB RAM, running Ubuntu 18.04 LTS. Each fuzzer ran for 24 hours, with the experiment repeated five times to obtain average results.

B. The Filtering Capability on Loops

It is proposed that fine-grained coverage is likely to draw the fuzzer's attention more towards loop structures, while MPS-Fuzz can mitigate this phenomenon. This section fevaluates the focus of different fuzzers on loop structures by statistically comparing the number of seeds captured solely due to varying loop iterations, as a proportion of all executed paths. Specifically, in the implementation, we re-execute all the execution paths discovered by each fuzzer, excluding those introduced by new edges or new combinations of adjacent conditional branches, and retaining only the paths introduced by different loop iterations. For each fuzzer, we calculate the proportion of paths triggered solely by varying loop iterations out of the total paths on different targets. The results are shown in Fig. 4. Due to the inability to compile the targets within FuzzBench, PathAFL's performance on the FuzzBench platform was not demonstrated. From the results depicted in the figure, it is evident that the proportion of paths attributable solely to loop structures in AFL is quite low. Regarding the N-gram method, as N increases, a higher proportion of paths are introduced by loop structures, with a marked increase observed when N is set to 4 and 8, where more than half of the new paths are induced by varying loop iterations. This effect is not pronounced when N is set to 2, likely because the increase from 2 to 3 in the number of basic blocks hashed is too minor, resulting in a degree similar to that of AFL. In the case of PathAFL, the proportion of loop paths is also high, falling between the levels observed when N is set to 4 and 8. Combining Fig. 4 with the data from Table II in the subsequent text, it can be inferred that the majority of the multiple times more new paths discovered by N-gram and PathAFL compared to AFL are introduced by different loop iterations. This indicates that current fine-grained coverage methods, i.e., N-gram and PathAFL, both overlook the impact brought about by loop structures. In contrast, the MPS-Fuzz method, which identifies paths related to loop structures, consistently finds such paths to be below 20% of the total, only slightly higher than AFL. This suggests that the loop-header-based filtering approach of MPS-Fuzz is effective.

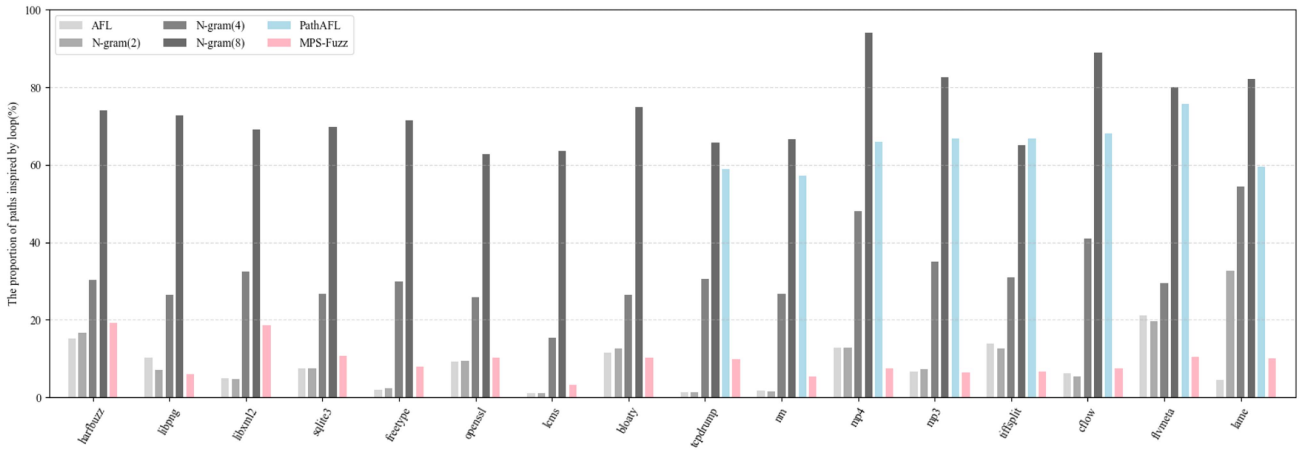

Fig. 4. The proportion of execution paths inspired solely by loop structures to all captured paths. AFL, N-gram(2), and MPS-Fuzz maintain low proportions, while N-gram(4), N-gram(8), and PathAFL show significant increases. Due to PathAFL's lack of support for LLVM instrumentation, it incorporates data from only 8 targets.

TABLE II
EDGE COVERAGE AND PATHS WITH DIFFERENT FUZZERS

Program	AFL		N-gram(2)		N-gram(4)		N-gram(8)		PathAFL		MPS-Fuzz	
	Edges	Paths	Edges	Paths	Edges	Paths	Edges	Paths	Edges	Paths	Edges	Paths
harfbuzz	**35553**	13269	34963	12090	34878	14422	34321	43000	–	–	35016	16100
libpng	2471	1709	3015	1785	3015	2447	3002	6080	–	–	**3021**	2450
libxml2	13363	8976	**16633**	10755	16214	16189	16030	36000	–	–	16436	19600
sqlite3	**26103**	8116	24917	7542	22125	8785	20709	16000	–	–	**26103**	10900
freetype	9024	8052	8946	7876	9031	12700	10676	32600	–	–	**11449**	14800
openssl	14634	1920	14628	1937	15211	2426	15257	5283	–	–	**15270**	2373
lcms	872	98	**931**	116	**931**	174	**931**	417	–	–	924	182
bloaty	10152	1027	10152	1052	10171	1330	**10191**	5989	–	–	10165	1442
tcpdump	19117	9671	19117	10159	19484	15933	17413	34300	18403	22096	**19694**	18000
nm	6082	3098	6147	3127	6075	5031	6029	13500	5957	7313	**6160**	5751
mp4	3061	1043	3061	947	3021	1474	3716	52000	3100	3967	**4011**	4926
mp3	1868	1439	1868	1359	1868	2369	1815	11200	1828	4115	**1933**	2616
tiffsqlit	2215	906	2202	970	2182	1529	2956	7079	2897	2623	**2962**	2597
cflow	**2248**	1039	**2248**	990	**2248**	1853	2235	12800	2235	3410	**2248**	1698
flvmeta	603	472	603	512	603	667	603	4176	603	2023	603	1067
lame	4470	1850	4443	1836	4476	3792	**4509**	11900	4496	5798	4470	2446
Average Increasing	-	-	+2.9%	+2.0%	+2.3%	+53.6%	+5.6%	+716.4%	+3.0%	+211.0%	**+9.6%**	+92.8%

In some targets (e.g., libxml2, tcpdump), MPS-Fuzz captures a more obvious number of loop-related paths compared to AFL. This is because the fuzzer's path retention rule is not solely based on new hits in the bitmap, but also related to the increase in hit buckets and the occurrence of crashes. Consequently, the number of captured loop-related paths is also higher than expected. Nonetheless, MPS-Fuzz demonstrates a obvious capability in filtering loop-related paths compared to existing fine-grained methods.

C. *Effectiveness in Increasing Edge Coverage*

Edge coverage, as a widely used metric, is employed by almost all fuzzers to assess their vulnerability discovery capabilities. This section demonstrates the performance of the proposed method compared to others in terms of edge coverage. Additionally, it compares the number of paths discovered by different methods. The edge coverage and path counts of different fuzzers across various programs are presented in Table II. Among them, the number of paths corresponds to the number of seeds.

Fine-grained coverage methods naturally have a significant advantage in the number of paths discovered compared to edge coverage. As shown in Table II, each fine-grained fuzzer significantly exceeds AFL in the number of paths, especially N-gram(8), with an average increase of up to 14x. However, in practice, these additional paths often result in limited edge coverage improvements and can lead the fuzzing process into locally complex structures. Therefore, it is crucial to focus on improving edge coverage rather than simply increasing the number of paths. Table II indicate that all fine-grained fuzzers outperform AFL in edge coverage. MPS-Fuzz shows the most significant improvement, with an average increase of 9.6%, while other methods only show an increase under 5%. In the 16 programs, the MPS-Fuzz method achieves the highest edge coverage in 10 of them, and in the other 6, it was close to the highest coverage rate. Moreover, in the projects "$libpng$", "$libxml2$", "$freetype$", "$mp4$", and "$tiffsqlite$", the MPS-Fuzz method shows a significant improvement in edge coverage compared to AFL. In contrast, the N-gram and PathAFL methods even have lower coverage rates than AFL in several projects, leading to limited overall coverage rate improvement. In our exploration, this is mainly due to the other fine-grained coverage methods lacking effective filtering mechanisms when capturing seeds, thus spending too much computational power on loop structures.

On the other hand, the excessive number of paths discovered by fine-grained methods does not consistently contribute to edge coverage improvement in certain programs. Although N-gram(8) captures a considerable number of paths (14x more than AFL), on targets like "$sqlite3$" and "$tcpdump$", the edge coverage of N-gram(8) even falls notably below that of AFL.

D. Effectiveness in Discovering Bugs

The works [17], [22] argues that high code coverage does not necessarily correlate directly with vulnerability discovery. To comprehensively evaluate the performance of the fuzzers, the experiments compare the number of bugs discovered by different fuzzers on various targets. Significantly, AFL-based fuzzers exhibit high redundancy in their unique crashes. As mentioned in some previous studies [23], [24], deduplicating crashes based on the call stack is an essential and efficient method. In this study, we use GDB and ASAN to debug each crash, excluding crashes related to input errors and excessive memory allocations. These crashes result from fuzzer inputs that do not conform to syntactic and semantic rules, rather than from bugs. And the first five frames of the call stack sequence were used as deduplication criteria (here, specific standard library calls will be excluded), providing a more stringent filter for crashes. Since on some targets, several fuzzers based on AFL could not trigger bugs, the paper only shows the number of triggered bugs on some programs in Table III.

Table III indicates that, apart from N-gram(2), all fine-grained coverage methods show an improvement in the number of bugs compared to AFL. Notably, the MPS-Fuzz demonstrates the most significant increase, with an overall improvement of 25.7%. We speculate that fine-grained coverage benefits vulnerability mining because it can discover more bugs. The performance of

TABLE III
BUGS WITH DIFFERENT FUZZERS

Program	AFL	N-gram (2)	N-gram (4)	N-gram (8)	PathAFL	MPS -Fuzz
mp4	3	1	5	1	5	**6**
tcpdump	32	34	34	34	33	**36**
mp3	6	5	**8**	6	3	6
cflow	8	8	**9**	8	7	**9**
tiffsplit	5	4	5	**9**	6	7
flvmeta	8	5	5	9	9	**11**
lame	8	7	9	10	10	**13**
Total	70	64 (-8.5%)	75 (+7.1%)	77 (+10.0%)	73 (+4.3%)	88 (+25.7%)

N-gram(2) falls short of AFL, possibly due to the minimal increase in the number of basic blocks undergoing hash operations from 2 to 3, which results in the minimal ability of the fuzzer to discover additional program paths. We argue that fine-grained coverage inevitably tends to bias the fuzzing process toward the loop structures in the program. These loop structures are not conducive to discovering bugs as the fuzzer spends more time on nearly identical loop structures in the call stack. However, MPS-Fuzz covers all paths within multiple MPS units while strategically excluding basic blocks within loop structures through selective instrumentation. The approach mitigates the issue of becoming trapped in loop structures, thereby achieving better performance in the number of bugs.

Moreover, MPS-Fuzz has discovered a stack-overflow type vulnerability in Xpdf, identified as CVE-2024-4568. The appendix also provides specific details of additional unknown bugs without CVE IDs that were discovered by MPS-Fuzz. Next, we present a detailed case study of this vulnerability to demonstrate the advantage of MPS-Fuzz in bug discovery. We first observed that the call chain triggering this vulnerability is: "$Stream.cc:791 < Object.cc:92 < XRef.cc:1218 < Object.h:263 < PSOutputDev.cc:2041 < PSOutputDev.cc:2044$". Since neither AFL nor N-gram triggered this bug, we examined their code coverage throughout the fuzzing process. The main reason is that both lack coverage of the conditional path at "$Object.h:263 < PSOutputDev.cc:2041 < PSOutputDev.cc:2044$" (as shown in Fig. 5, trigger path is marked in red color). First, we observe that PSOutputDev.cc:2041 is a typical MPS-unit: it has two outgoing edges and is reachable from many callers (i.e., many functions invoke this site). For edge coverage of AFL, the explosion of possible edge combinations makes it highly probable that the single critical path leading to the bug is simply forgotten. Second, several entry-points repeatedly call structures inside PSOutputDev.cc in a cyclic way. The N-gram therefore keeps generating seeds that differ only in loop count; the resulting seed inflation causes the scheduler to waste most of its time on uninteresting inputs, drastically lowering the chance of exposing the decisive path. Consequently, MPS-Fuzz succeeded in exposing it for the two reasons outlined above, while AFL and N-gram both missed the vulnerability.

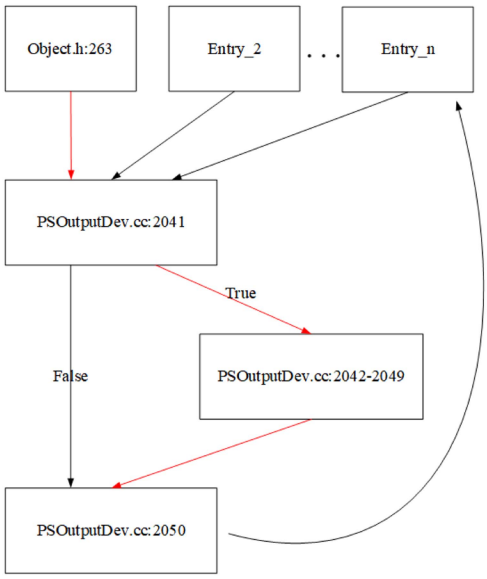

Fig. 5. CFG of CVE-2024-4568.

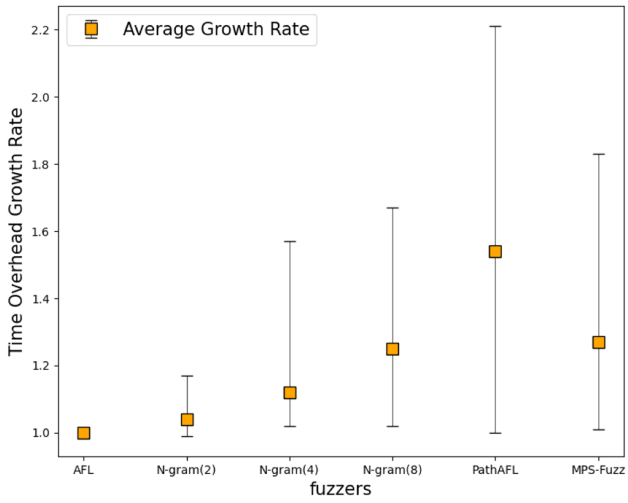

Fig. 6. Time overhead increasing compared to AFL. The vertical axis represents the execution time of each fuzzer relative to AFL, with AFL set as the baseline at 1. Each bar displays the average, maximum, and minimum values. In calculating these values, PathAFL, due to its lack of support for LLVM instrumentation, includes data from only 8 targets, while the others include data from all 16 targets.

E. Time Overhead and Collisions Evaluation

To achieve finer-grained coverage, fuzzer requires additional operations, such as adding instrumentation code, which may incur extra time overhead and hash collisions. In this subsection, we evaluate these costs compared to N-gram and PathAFL.

To objectively evaluate time overhead, we aggregate the seeds produced by different fuzzers over 24 hours as input examples. The average time taken by each fuzzer to run these combined seeds is used as an evaluation of time overhead, as illustrated in Fig. 6.

The figure illustrates that to collect more execution paths, each fine-grained fuzzer incurs additional time overhead compared to

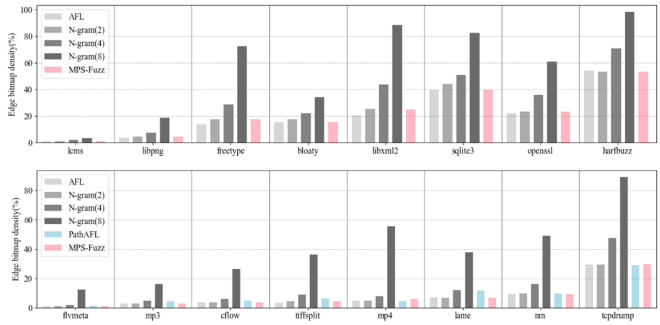

Fig. 7. Increasing of edge bitmap density compared to AFL. Due to PathAFL's lack of support for LLVM instrumentation, it incorporates data from only 8 targets.

AFL. The reasons for each fuzzer are analyzed as follows: For N-gram, as the granularity becomes finer (i.e., N increases), more hash operations are performed at each basic block, resulting in increasing time overhead. Due to hashing the entire sequence of basic blocks and the need to compare branch block information at runtime, PathAFL has a relatively high time overhead. The time overhead for MPS-Fuzz is primarily due to the additional instrumentation code required to record the multi-input vector. Statistical data show that MPS-Fuzz has an average additional overhead of 27% compared to AFL, with a maximum of 80% on certain targets. This magnitude is close to N-gram(8) among the other fine-grained fuzzers compared, better than PathAFL, but worse than N-gram(2) and N-gram(4). Considering the enhancement in vulnerability mining capabilities brought by MPS-Fuzz, the increase in time overhead should be acceptable.

Fine-grained fuzzers discovering additional paths require extra data for recording and retrieval, which may lead to additional collisions in edge bitmap hashing. The collision rate is positively correlated with edge bitmap density (as discussed in the BigMap [25], and further analyzed from a probabilistic perspective in Section 6 of this paper). Fig. 7 illustrates the bitmap density for different fuzzers across various targets. In cases where almost no new edge is discovered compared to AFL, N-gram exhibits a noticeable increase in map density. This is because the N-gram directly utilizes the edge bitmap to record additional paths. PathAFL uses few edge bitmaps to record extra path values, thus a slight increase in edge bitmap density increasing is observed (e.g., "$cflow$"", "$tiffsplit$", "$lame$"). In contrast, MPS-Fuzz does not rely on edge bitmaps at all to record MPS unit IDs, resulting in minimal impact on edge bitmap density. The growth in density of MPS-Fuzz on individual targets, e.g., "$freetype$", "$libxml2$", is attributed to the discovery of new edges that AFL did not find.

F. Ablation Experiments

This subsection primarily conducts ablation experiments on the parameters of MPS-Fuzz. On our benchmark suite, we illustrate the effects of the ablation process in terms of discovered bug targets, focusing on paths, edges, and bugs. On the one hand, the length of the multi-input vector in MPS-Fuzz can be specified, where a longer length corresponds to finer granularity

TABLE IV
ABLATION FOR DIFFERENT MULTI-INPUT VECTOR LENGTHS

Program	MPS-Fuzz(2)			MPS-Fuzz(3)			MPS-Fuzz(5)		
	Paths	Edges	Bugs	Paths	Edges	Bugs	Paths	Edges	Bugs
tcpdump	14300	19281	32	18000(+25.9%)	19694(+2.1%)	**36**	25000(+74.8%)	**19864**(+3.0%)	34
mp4	2725	3899	2	2926(+7.4%)	**4011**(+2.9%)	6	16800(+516%)	3670(-5.9%)	**11**
mp3	1980	**1933**	**7**	2616(+32.1%)	1861(-3.7%)	6	5846(+195%)	1861(-3.7%)	3
tiffsqlit	1548	2870	6	2579(+66.6%)	**2962**(+3.2%)	7	5814(+275%)	2903(+1.1%)	**9**
cflow	1284	2248	7	1698(+32.2%)	2248(0%)	**9**	4297(+234%)	**2261**(+0.6%)	9
flvmeta	839	603	8	1067(+27.2%)	603(0%)	**11**	2206(+162%)	603(0%)	9
lame	1987	4325	10	2446(+23.1%)	4371(+1.1%)	**13**	4668(+135%)	4338(+0.3%)	11
Average Increasing	-	-	-	+30.1%	+0.8%	+22.2%(total)	+228.0%	-0.6%	+19.4%(total)

TABLE V
ABLATION FOR LOOP STRUCTURE

Program	MPS-Fuzz			MPS-Fuzz-loop		
	Paths	Edges	Bugs	Paths	Edges	Bugs
tcpdump	18000	19694	**36**	22600(+25.7%)	**20251**(+2.8%)	34
mp4	2926	**4011**	**6**	8416(+187%)	3408(-15.0%)	2
mp3	2616	1861	**6**	3270(+25.0%)	**1868**(+0.3%)	6
tiffsqlit	2579	**2962**	**7**	3603(+39.7%)	2949(-0.4%)	5
cflow	1698	**2248**	**9**	2174(+28.1%)	2241(-0.3%)	7
flvmeta	1067	603	**11**	1296(+21.4%)	603(0%)	8
lame	2446	4371	**13**	2813(+15.0%)	**4384**(+0.3%)	9
Average Increasing	-	-	-	+30.1%	-1.8%	-19.3%(total)

TABLE VI
INTEGRATION OF HAVOC-MAB AND MPS-FUZZ

Program	Havoc-mab			Havoc-mab+MPS		
	Paths	Edges	Bugs	Paths	Edges	Bugs
tcpdump	11300	20565	36	21100(+86.7%)	**22426**(+9.1%)	**45**
mp4	1331	3218	1	3917(+194%)	**3768**(+17.1%)	**3**
mp3	1480	**1868**	4	2615(+76.7%)	1855(-0.7%)	4
tiffsqlit	923	2956	**8**	2825(+206%)	**3028**(+2.4%)	6
cflow	1011	2248	7	1710(+69.2%)	**2248**(0%)	**9**
flvmeta	451	603	**5**	968(+115%)	603(0%)	4
lame	2053	4450	10	2560(+24.7%)	**4476**(+0.6%)	**11**
Average Increasing	-	-	-	+110%	+4.1%	+15.5%(total)

of coverage. We compare this length by taking values of 2, 3, and 5, denoted as MPS-Fuzz(2), MPS-Fuzz(3), and MPS-Fuzz(5), respectively, as shown in Table IV. Percentage improvement relative to MPS-Fuzz(2) is indicated in parentheses and last line. The data shows that as the multi-input vector increases, there is a noticeable increase in the number of paths captured by fuzzers, indicating a gradual refinement in coverage granularity. Regarding edge coverage, the differences among the three are minimal. In terms of bug count, there is not much difference between MPS-Fuzz(3) and MPS-Fuzz(5), but they both show a significant advantage over MPS-Fuzz(2). Our analysis suggests that one MPS unit can include a different number of basic blocks, and with MPS units set at 3, the granularity is already sufficiently fine. The length set to 2 is still not fine-grained enough. However, having too many MPS units increases time overhead and dilutes the probability of bugs capture due to the excessive number of seeds. Therefore, considering all three metrics, setting the multi-input vector to 3 is more appropriate.

On the other hand, MPS-Fuzz mitigates the influence of loop structures. We conduct a comparison by removing this operation to observe its contribution, denoted as MPS-Fuzz-loop, as shown in Table V. Percentage improvement relative to MPS-Fuzz is indicated in parentheses and last line. The data indicates that loop structures contribute to an additional average of 30.1% paths, yet result in decreased average edge coverage and bug counts. While the impact on edge coverage is minimal, the overall decline in bugs is significant at 19.4%, which substantially affects the fuzzing capability. Therefore, it is essential to consider the negative implications of loop structures for fine-grained coverage.

G. Scalability of MPS-Fuzz

To evaluate the scalability of MPS-Fuzz, we introduce MPS unit guidance into Havoc-mab [26], denoted as Havoc-mab+MPS. Havoc-mab models the seed mutation process as a multi-armed bandit and selects suitable mutation methods through reinforcement learning. In principle, Havoc-mab and MPS-Fuzz are orthogonal. In implementation, both are based on AFL. Using the same experimental setup as before, the comparison is shown in Table VI. The data reveals that with the inclusion of the MPS-Fuzz mechanism, the average edge coverage of Havoc-mab increases by 4.1%, and the overall discovery of bugs surges by 15.5%. Particularly noteworthy is the performance on the "$tcpdump$" program, where the addition of the MPS-Fuzz results in an edge coverage of 34.22% and a bug count of 45, surpassing significantly the performance of all fuzzer variants mentioned in this paper. The experiment demonstrates that MPS-Fuzz has the potential to enhance other orthogonal methods.

V. DISCUSSION

Orthogonality of MPS-Fuzz to some sensitive coverage: In principle, MPS-Fuzz, along with the N-gram and PathAFL compared in the paper, all aim to identify execution paths from the perspective of CFG. In contrast, some sensitive coverage techniques utilize non-CFG information for guidance, for instance, calling context [27], heap memory usage information [28], or dataflow information [29]. Consequently, MPS-Fuzz also holds the potential for synergistic enhancement with these non-CFG coverage methods.

Limitation and future work: Although MPS-Fuzz has demonstrated its capability for vulnerability discovery. There are still some limitations. Firstly, although the paper has reduced the capture of homogeneous paths by preprocessing methods that exclude loop structures, it has yet to employ specialized seed selection mechanisms as post-processing to refine the number of seeds further. Future work could explore using adaptive scheduling method [30], [31] to optimize seed selection strategies. Secondly, MPS-Fuzz incurs acceptable but still noticeable time overhead compared to AFL. Future efforts could integrate lightweight instrumentation techniques (e.g., Instrim [32] and Zeror [33]) to mitigate time overhead.

VI. Related Work and Background

The success of grey-box fuzzing in uncovering vulnerabilities is closely tied to coverage-guided fitness. This section primarily focuses on discussing techniques related to coverage.

Sensitive coverage: The coverage scale is crucial to evaluate the favorability of specific test cases within a large set. Various techniques have enhanced edge coverage to uncover more vulnerabilities. For instance, some works [27], [34], [35] adds calling context to the edge coverage map, Memlock [36] and HTFuzz [28] adds memory usage information, and some works [37], [38], [39], [40] integrate lightweight taint analysis. DATAFLOW [29] solely focuses on data flow for guidance. Fine-grained coverage methods achieve more sensitive coverage solely from the control flow perspective. Previous works [14], [15] also mention the "path lost" problem, describing the phenomenon but not explaining the underlying principles. Some fine-grained fuzzers have proposed specific approaches to mitigate the issue of "path lost". N-gram expands hashing to more basic blocks, but this increases time overhead and hash collisions. PathAFL hashes all basic blocks along the execution path, optimizing with selective instrumentation and modified hashing. Although it achieves path coverage, selective instrumentation has the probability of missing critical basic blocks, such as the MPS basic blocks mentioned in our study. Missing these critical basic blocks would decrease the impact of path coverage outside of edge coverage. Ankou [18] still utilizes edge coverage, but they take into account the hit count for each edge. So Ankou primarily identifies more loops and may be ineffective in addressing the "path lost" issue caused by multiple conditional statements. Ankou refactored the entire fuzzer using the Go language, which differs significantly from AFL-based methods, making it challenging to control variables in comparing performance with other fine-grained mechanisms. As discussed above, N-gram and PathAFL address the "path lost" issue more directly and comprehensively. Therefore, the paper compares the proposed method with the AFL, N-gram and PathAFL in the experimental section.

Relationship between map density and collision: As the most well-known and widely used grey-box fuzzer, AFL introduced the concept of edge coverage and the data structure bitmap for recording edge hits. This contribution has been widely adopted and discussed in subsequent research [20], [27], [28], [37]. AFL captures information about the executed edges during program runtime through compile-time instrumentation, storing this data in a 64 KB shared memory known as the bitmap. The edge IDs are obtained by hashing the IDs of two adjacent basic blocks on the execution path, using a hash operation involving a $shift$ and a xor, that is, $(prev\_loc >> 1) \oplus cur\_loc$. This approach can lead to hash collisions, where different edges share the same hash value. CollAFL [41] first addresses this issue by employing a method that replaces collided values during lookups to eliminate hash collisions. However, this approach incurs significant time overhead. Bigmap [25] proposes using a larger-sized bitmap to alleviate collisions, and it introduces a two-level bitmap to mitigate the time overhead. Nevertheless, this approach only partially eliminates collisions. Although mitigating hash collisions is not the paper's primary focus, the finer-grained coverage, due to its ability to discover more program paths, may lead to a higher collision rate. Collision rates correlate with bitmap density, where higher density indicates increased collision likelihood [25]. It utilizes a probability model to explore hash collision rates further. In edge coverage, map hits align with edge IDs, while in finer-grained methods, they correspond to basic block sequence IDs. As these IDs are derived from randomly distributed basic block IDs, map hits across different coverage granularity also exhibit random distribution. We can obtain the formula for the mathematical expectation of collision rate, denoted as $E_{(CR)}$:

$$E_{(CR)} = 1 - \frac{H}{n}\left(1 - \left(\frac{H-1}{H}\right)^{n}\right) \tag{4}$$

Here, H is the size of bitmap, n/H is map density. Equation (4) allows us to calculate hash collision rates based on bitmap density. However, real-world scenarios may slightly differ from the computed results, as the formula provides the mathematical expectation of collision rate.

Formula (4) is also provided in the context of Bigmap and document of AFL++, and the derivation process is essentially the "Balls in Bins" algorithm.

VII. Conclusion

The paper introduces the concept of MPS units, explaining the relationship between typical fuzzing coverage and program execution paths based on the topological structure of the program's CFG. Utilizing MPS units, the paper proposes an enhanced fine-grained coverage method called the MPS-Fuzz. Additionally, it employs a preprocessing method to exclude MPS units with loop structures, effectively filtering out homogeneous paths and optimizing the fuzzer's vulnerability detection performance. Compared to the SOTA methods N-gram and PathAFL, the proposed approach demonstrates edge coverage and bug discovery advantages. And MPS-Fuzz uncovered a CVE on real-world program. Furthermore, MPS-Fuzz can be easily combined with orthogonal methods, such as seed mutation techniques like havoc-mab, to enhance performance.

VII. Acknowledgment

The authors sincerely appreciate the anonymous reviewers for their valuable comments, which greatly contributed to improving the quality of this work.

References

[1] H. Green and T. Avgerinos, "GraphFuzz: Library api fuzzing with lifetime-aware dataflow graphs," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1070–1081.
[2] P. Chen, Y. Xie, Y. Lyu, Y. Wang, and H. Chen, "Hopper: Interpretative fuzzing for libraries," in *Proc. 2023 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1600–1614.
[3] R. Mahmood, J. Pennington, D. Tsang, T. Tran, and A. Bogle, "A framework for automated api fuzzing at enterprise scale," in *Proc. 2022 IEEE Conf. Softw. Testing, Verification Validation*, 2022, pp. 377–388.
[4] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Aflnet: A greybox fuzzer for network protocols," in *Proc. IEEE 13th Int. Conf. Softw. Testing, Validation Verification*, 2020, pp. 460–465.
[5] J. Li, S. Li, G. Sun, T. Chen, and H. Yu, "SNPSFuzzer: A fast greybox fuzzer for stateful network protocols using snapshots," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2673–2687, 2022.
[6] R. Zhong, Y. Chen, H. Hu, H. Zhang, W. Lee, and D. Wu, "Squirrel: Testing database management systems with language validity and coverage feedback," in *Proc. 2020 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 955–970.
[7] Y. Li, Y. Nie, and X. Kuang, "Fuzzing DBMS via NNLM," in *Proc. 7th IEEE Int. Conf. Data Sci. Cyberspace*, 2022, pp. 367–374.
[8] M. Zalewski, "American fuzzy lop, v2.52b," 2020. [Online]. Available: https://lcamtuf.coredump.cx/afl/
[9] R. Kersten, K. Luckow, and C. S. Păsăreanu, "Poster: AFL-based fuzzing for java with kelinci," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 2511–2513.
[10] M. Wu, M. Lu, H. Cui, J. Chen, Y. Zhang, and L. Zhang, "Jitfuzz: Coverage-guided fuzzing for JVM just-in-time compilers," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 56–68.
[11] W. Li, J. Ruan, G. Yi, L. Cheng, X. Luo, and H. Cai, "POLYFUZZ: Holistic greybox fuzzing of multi-language systems," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 1379–1396.
[12] J. Fu, S. Xiong, N. Wang, R. Ren, A. Zhou, and B. K. Bhargava, "A framework of high-speed network protocol fuzzing based on shared memory," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2779–2798, Jul./Aug. 2024.
[13] J. Wang, Y. Duan, W. Song, H. Yin, and C. Song, "Be sensitive and collaborative: Analyzing impact of coverage metrics in greybox fuzzing," in *Proc. 22nd Int. Symp. Res. Attacks, Intrusions Defenses*, 2019, pp. 1–15.
[14] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–36, 2022.
[15] S. Yan, C. Wu, H. Li, W. Shao, and C. Jia, "Pathafl: Path-coverage assisted fuzzing," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, 2020, pp. 598–609.
[16] "Fuzzbench: Fuzzer benchmarking as a service," 2020. [Online]. Available: https://github.com/google/fuzzbench
[17] Y. Li et al., "Unifuzz: A holistic and pragmatic metrics-driven platform for evaluating fuzzers," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2777–2794.
[18] V. J. Manès, S. Kim, and S. K. Cha, "Ankou: Guiding grey-box fuzzing towards combinatorial difference," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 1024–1036.
[19] Website, "Honggfuzz," 2018. Accessed: 04, 2018. [Online]. Available: http://honggfuzz.com/
[20] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "{AFL++ }: Combining incremental steps of fuzzing research," in *Proc. 14th USENIX Workshop Offensive Technol.*, 2020.
[21] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "{AddressSanitizer}: A fast address sanity checker," in *Proc. 2012 USENIX Annu. Tech. Conf.*, 2012, pp. 309–318.
[22] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 2123–2138.
[23] A. K. Joshy and W. Le, "Grouping fuzzed crashes based on fault signatures," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–12.
[24] T. Blazytko et al., "{AURORA}: Statistical crash analysis for automated root cause explanation," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 235–252.
[25] A. Ahmed, J. D. Hiser, A. Nguyen-Tuong, J. W. Davidson, and K. Skadron, "Bigmap: Future-proofing fuzzers with efficient large maps," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2021, pp. 531–542.
[26] M. Wu et al., "One fuzzing strategy to rule them all," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1634–1645.
[27] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *Proc. 2018 IEEE Symp. Secur. Privacy*, 2018, pp. 711–725.
[28] Y. Yu et al., "Htfuzz: Heap operation sequence sensitive fuzzing," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–13.
[29] A. Herrera, M. Payer, and A. L. Hosking, "Dataflow: Toward a data-flow-guided fuzzer," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 5, pp. 1–31, 2023.
[30] T. Yue et al., "{EcoFuzz }: Adaptive { Energy-Saving } greybox fuzzing as a variant of the adversarial { Multi-Armed} bandit," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 2307–2324.
[31] S. Li and Z. Su, "Accelerating fuzzing through prefix-guided execution," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 1–27, 2023.
[32] C.-C. Hsu, C.-Y. Wu, H.-C. Hsiao, and S.-K. Huang, "Instrim: Lightweight instrumentation for coverage-guided fuzzing," in *Proc. Symp. Netw. Distrib. Syst. Secur., Workshop Binary Anal. Res.*, 2018, pp. 1–7.
[33] C. Zhou, M. Wang, J. Liang, Z. Liu, and Y. Jiang, "Zeror: Speed up fuzzing with coverage-sensitive tracing and scheduling," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 858–870.
[34] Z.-M. Jiang, J.-J. Bai, K. Lu, and S.-M. Hu, "Fuzzing error handling code using {Context-Sensitive} software fault injection," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 2595–2612.
[35] Z.-M. Jiang, J.-J. Bai, K. Lu, and S.-M. Hu, "Context-sensitive and directional concurrency fuzzing for data-race detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–18.
[36] C. Wen et al., "Memlock: Memory usage guided fuzzing," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 765–777.
[37] S. Gan et al., "{GREYONE}: Data flow sensitive fuzzing," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 2577–2594.
[38] J. Liang et al., "PATA: Fuzzing with path aware taint analysis," in *Proc. 2022 IEEE Symp. Secur. Privacy*, 2022, pp. 1–17.
[39] J. Kukucka, L. Pina, P. Ammann, and J. Bell, "Confetti: Amplifying concolic guidance for fuzzers," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 438–450.
[40] Z. Jia, C. Yang, X. Zhao, X. Li, and J. Ma, "Design and implementation of an efficient container tag dynamic taint analysis," *Comput. Secur.*, vol. 135, 2023, Art. no. 103528.
[41] S. Gan et al., "Collafl: Path sensitive fuzzing," in *Proc. 2018 IEEE Symp. Secur. Privacy*, 2018, pp. 679–696.

Ximing Fan received the master's degree from the Nanjing University of Posts and Telecommunications in 2022. He is currently working toward the PhD degree with the School of Cyber Science and Engineering, Sichuan University, China. His research interests include software vulnerability mining, static program analysis, and artificial intelligence security.

Yong Fang received the PhD degree from Sichuan University, Chengdu, China, in 2010. He is currently a professor with the School of Cyber Science and Engineering, Sichuan University. His research interests include network security, web security, Internet of Things, Big Data, and artificial intelligence.

Peng Jia received the BEng and PhD degrees from Sichuan University, Chengdu, China, in 2012 and 2019, respectively. He is currently an assistant research fellow with the School of Cyber Science and Engineering, Sichuan University. His research interests include binary security, network science, and malware propagation.

Hongwei Li (Fellow, IEEE) received the PhD degree from the University of Electronic Science and Technology of China in 2008. He is currently a professor with the School of Computer Science and Engineering, University of Electronic Science and Technology of China. His research interests include network security and applied cryptography. Dr. Li is an associate editor for *IEEE Internet of Things Journal*. He is/was the technical symposium co-chair of IEEE ICC 2024, ACM TUR-C 2019, and IEEE GLOBECOM 2015. He won Best Paper Awards of IEEE ICPADS 2020 and IEEE MASS 2018. He is the vice chair(conference) of IEEE ComSoc CIS-TC. He is the distinguished lecturer of IEEE Vehicular Technology Society.

Xi Peng received the MEng degree from the China Academy of Telecommunications Technology, Beijing, China, in 2020. He is currently working toward the PhD degree with the School of Cyber Science and Engineering, Sichuan University, Chengdu, China. His research interests include binary security, system security, and artificial intelligence.

Yijia Xu received the PhD degree from Sichuan University, Chengdu, China, in 2024. He is currently an assistant research fellow with the School of Cyber Science and Engineering, Sichuan University. His current research interests include web security, cyber confrontation, and graph neural network.

Qinying Wang received the BS degree from the College of Computer Science and Electronic Engineering, Hunan University in 2018, and the PhD degree from the College of Computer Science and Technology, Zhejiang University in 2024. Her research interests include IoT security, fuzzing, and AI-assisted security.

Binbin Zhao received the BS degree from the College of Computer Science, Zhejiang University in 2018, and the master's and PhD degrees in electrical and computer engineering from Georgia Tech in 2022 and 2023, respectively. He is currently a research faculty with the School of Electrical and Computer Engineering, Georgia Tech. His research interests include IoT security, fuzzing, and data-driven security.

Shouling Ji (Member, IEEE) received the PhD degree in electrical and computer engineering from the Georgia Institute of Technology, and the second PhD degree in computer science from Georgia State University. He was a research intern with IBM T.J. Watson Research Center. He is currently a Qiushi distinguished professor with the College of Computer Science and Technology, Zhejiang University. His research interests include data-driven security and privacy, AI security, software and system security. He is a member of ACM and senior member of CCF. Shouling was the recipient of the 2012 Chinese Government Award for Outstanding Self-Financed Students Abroad and 10 Best/Outstanding Paper Awards, including ACM CCS 2021.